

Llenguatges de marques i sistemes de gestió d'informació

Introducció actualitzada · CFGS Desenvolupament d'Aplicacions Web

2026

De SGML a JSON, YAML i la web actual

Objectiu

Entendre, comparar i triar formats

Metodologia

Exemples + activitats curtes



Què hauríem de saber en acabar la introducció?

- Distingir llenguatge de marques, format de serialització, llenguatge de configuració i llenguatge de programació.
- Reconèixer estructura, dades, metadades, etiquetes, atributs, claus, valors i jerarquia.
- Comparar HTML, XML, JSON, YAML, TOML i Markdown segons el problema que resolen.
- Entendre per què HTML i XML continuen vius, però comparteixen espai amb formats més moderns.
- Llegir i modificar exemples reals de web, APIs, GitHub Actions, Docker Compose i configuració.

1 · Què és «marcar» informació?



Abans de parlar de formats, separem contingut, estructura, presentació i significat.



Un llenguatge de marques afegeix informació al contingut

Text sense estructura

Oferta: Portàtil

Preu: 899 €

Disponible: sí

Text amb marques

```
<producte>
```

```
<nom>Portàtil</nom>
```

```
<preu moneda="EUR">899</preu>
```

```
<disponible>true</disponible>
```

```
</producte>
```

Quatre capes que convé no confondre

Contingut

«899 €»

La informació que volem comunicar.

Estructura

<preu>899</preu>

Què és cada part i com s'organitza.

Presentació

color, mida, posició

Com es mostra a l'usuari.

Semàntica

preu + moneda

Quin significat té per a persones i màquines.

Activitat 1 · Marca el menú

Aplicació immediata del que acabem de veure

Tens aquest text: «Entrepà vegetal · 5,50 € · al·lèrgens: gluten, ou». Proposa una estructura que permeti a una aplicació identificar nom, preu i al·lèrgens.

Temps

8 min

Format

Parelles

Resultat

3–5 línies de codi

No tot el text estructurat és un llenguatge de marques

Marques

HTML · XML · SVG

Usen elements/etiquetes per descriure estructura o semàntica.

Serialització

JSON · YAML

Representen dades estructurades per guardar-les o intercanviar-les.

Configuració

YAML · TOML · JSON

Sovint descriuen opcions i paràmetres d'un sistema.

Text lleuger

Markdown

Afig una sintaxi mínima per estructurar documentació.

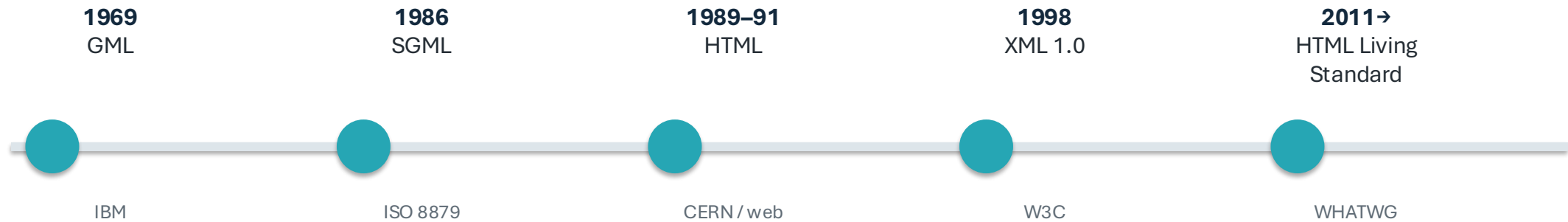
2 · D'on venim?



Una història curta: GML → SGML → HTML i XML. Només el necessari per entendre el present.



De documents impresos a la web actual



Idea clau

La història explica les decisions de disseny, però no ha de ser el centre del mòdul.

SGML: important històricament, secundari en la pràctica actual

Què aporta

És un metallenguatge: permet definir llenguatges de marques i formalitzar la seua estructura.

Per què importa

HTML i XML s'entenen millor sabent que naixen d'aquesta tradició de documents estructurats.

Què fem al curs

Una diapositiva i un exemple. No cal dedicar-li el mateix pes que a HTML, XML o JSON.

3 · HTML hui



HTML ja no és «un document per mostrar text»: és el llenguatge semàntic central de la plataforma web.



HTML és estructura i semàntica; CSS és presentació

HTML SEMÀNTIC

```
<!doctype html>
<html lang="ca">
  <head>
    <meta charset="utf-8">
    <title>Notícies del centre</title>
  </head>
  <body>
    <main>
      <article>
        <h1>Jornada de portes obertes</h1>
        <time datetime="2026-09-20">20 de setembre</time>
      </article>
    </main>
  </body>
</html>
```

<main>

Contingut principal de la pàgina.

<article>

Unitat de contingut independent.

<time>

Aporta significat temporal llegible per màquines.

Del HTML presentacional al HTML semàntic

ABANS: APARENÇA DINS DE L'HTML

```
<body bgcolor="#ffffff">
  <font size="+2"><b>Notícies</b></font>
  <br><br>
  <font color="blue">20/09/2026</font>
</body>
```

ARA: SIGNIFICAT EN HTML

```
<main>
  <article>
    <h1>Notícies</h1>
    <time datetime="2026-09-20">20/09/2026</time>
  </article>
</main>
```

Presentació → CSS · Comportament → JavaScript · Estructura i semàntica → HTML

HTML és un Living Standard

Ja no té sentit ensenyar «HTML 4 → HTML 5» com si HTML quedara congelat en una versió.

Evolució contínua

L'especificació es manté i actualitza de manera contínua. Les funcionalitats entren a mesura que maduren.

Web moderna

Inclou semàntica, formularis, multimèdia, interacció i APIs relacionades amb documents i aplicacions.

Extensibilitat

Els Custom Elements permeten definir elements propis amb comportament, integrats amb el DOM.

Activitat 2 · Detectiu d'HTML

Aplicació immediata del que acabem de veure

Busca en una web real (premsa, botiga o centre educatiu) exemples de <header>, <nav>, <main>, <article>, <section>, <footer> i <time>. Quins usa bé? Quins substituiries?

Temps

15 min

Format

Grups de 3

Resultat

Captura + 5 conclusions

4 · XML: menys visible, però encara rellevant

No domina les APIs web modernes, però continua sent clau en documents, integració, estàndards i eco



XML: dades estructurades amb vocabulari extensible

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<alumnes curs="2026-27">
  <alumne id="a17">
    <nom>Aina</nom>
    <modul>LMSGI</modul>
    <nota>8.5</nota>
  </alumne>
</alumnes>
```

Ben format

Una arrel, etiquetes correctament niades, atributs entre cometes...

Vàlid

A més, compleix una gramàtica/esquema: DTD, XSD, RELAX NG...

Namespaces

Permeten combinar vocabularis XML sense col·lisions de noms.

On trobem XML de veritat?

SVG

Gràfics vectorials dins de la web i eines de disseny.

RSS / Atom

Sindicació de continguts, podcasts i feeds.

Office Open XML

DOCX, XLSX i PPTX són contenidors ZIP amb molts fitxers XML.

Maven

pom.xml descriu dependències i construcció en projectes Java.

Android

Encara és habitual en layouts, recursos i manifests.

SOAP / estàndards

Integració empresarial, administracions i protocols amb esquemes forts.

Validar és convertir una estructura en un contracte

FRAGMENT XSD

```
<xs:element name="nota">
  <xs:simpleType>
    <xs:restriction base="xs:decimal">
      <xs:minInclusive value="0"/>
      <xs:maxInclusive value="10"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

Sense esquema

«nota» pot contindre pràcticament qualsevol text.

Amb esquema

Podem exigir tipus, rangs, obligatorietat, ordre i cardinalitat.

Idea transferible

El mateix concepte reapareix en JSON Schema, OpenAPI i validació de configuracions.

Activitat 3 · Trenca l'XML

Aplicació immediata del que acabem de veure

A partir d'un XML correcte, crea quatre errors diferents: etiqueta mal tancada, dues arrels, atribut sense cometes i estructura que no complisca l'esquema. Classifica: «no ben format» o «no vàlid».

Temps

12 min

Format

Parelles

Resultat

4 errors + classificació

5 · Formats que l'alumnat veurà cada dia

JSON, YAML, TOML i Markdown no substitueixen HTML/XML: resolen altres problemes i conviuen amb el



JSON: el format quotidià de les APIs web

RESPOSTA D'UNA API

```
{  
  "id": 17,  
  "nom": "Aina",  
  "actiu": true,  
  "moduls": ["LMSGI", "Programació"],  
  "contacte": {  
    "email": "aina@example.org"  
  }  
}
```

Tipus simples

string · number · boolean · null

Tipus estructurats

objectes {} i arrays []

Fortalesa

Sintaxi mínima i suport pràcticament universal en llenguatges i APIs.

XML o JSON? Depén del problema

XML

- ✓ Documents mixtos i vocabularis extensibles
 - ✓ Namespaces
 - ✓ Ecosistema madur d'esquemes i transformacions
 - ✓ Estàndards empresarials/documentals
- Més verbós per a dades simples

JSON

- ✓ Natural per a objectes i arrays
 - ✓ Molt compacte
 - ✓ Ideal per a APIs web i JavaScript
 - ✓ Ecosistema modern enorme
- Menys adequat per a document mixt i namespaces

YAML: serialització llegible, molt usada per configurar

DOCKER-COMPOSE.YML

```
services:
  web:
    image: nginx:alpine
    ports:
      - "8080:80"
    environment:
      APP_ENV: development
```

On apareix

Docker Compose, Kubernetes, GitHub Actions, Ansible, CI/CD...

Avantatge

Molt llegible quan la jerarquia és moderada.

Perill

La indentació és significativa; petits canvis poden alterar l'estructura.

TOML: configuració explícita i previsible

PYPROJECT.TOML / CONFIGURACIÓ

```
[project]
name = "exemple-lmsgi"
version = "1.0.0"
```

```
[tool.app]
debug = true
ports = [8000, 8001]
```

Objectiu

Format mínim de configuració, fàcil de llegir i mapar a estructures clau-valor.

On apareix

pyproject.toml, Cargo.toml (Rust) i moltes eines modernes.

Comparació

Menys flexible que YAML, però sovint més previsible per a configuració.

Markdown: documentació com a text pla versionable

README.MD

```
# Pràctica 1
```

```
## Objectiu
```

```
Consumir una **API REST**.
```

```
```bash
```

```
curl https://api.exemple.org/alumnes
```

```
```
```

- [] Fer la petició
- [] Analitzar el JSON

GitHub / GitLab

README, issues, documentació i wikis.

Docs as code

La documentació pot passar per Git, revisions i CI com el codi.

Matís

Markdown no és un únic dialecte; CommonMark intenta fixar una base interoperable.

Activitat 4 · Una dada, quatre formats

Aplicació immediata del que acabem de veure

Representa la mateixa informació —un alumne amb nom, edat, mòduls i correu— en XML, JSON, YAML i TOML. Després respon: quin és més llegible? quin és més verbós? quin validaries amb més facilitat?

Temps

20 min

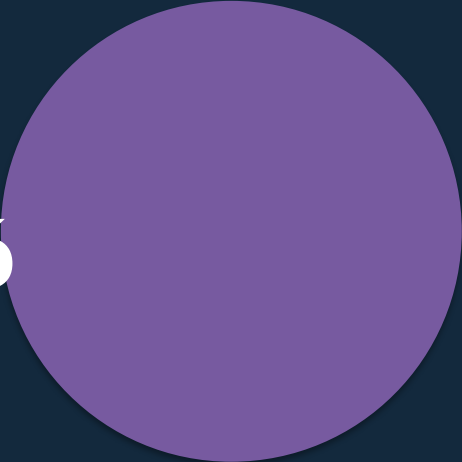
Format

Grups de 4

Resultat

Comparativa en una taula

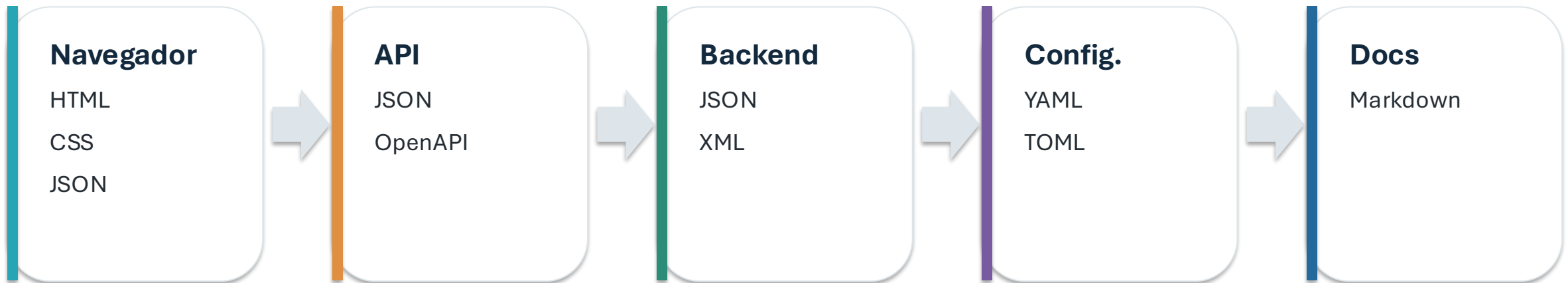
6 · De fitxers a sistemes d'informació



Els formats tenen sentit quan viatgen entre aplicacions, APIs, bases de dades, navegadors i eines.



El mateix sistema pot usar molts formats



Competència important no és memoritzar sintaxi: és saber identificar l'estructura i triar/transformar el format adequat

Exemple realista: una API REST

HTTP + JSON

```
GET /api/productes/42
Accept: application/json
```

```
HTTP/1.1 200 OK
Content-Type: application/json
```

```
{
  "id": 42,
  "nom": "Teclat mecànic",
  "preu": 79.90
}
```

Protocol

HTTP transporta la petició i la resposta.

Metadades

Content-Type indica com interpretar el cos.

Serialització

JSON representa les dades de la resposta.

JSON-LD: quan JSON també necessita significat compartit

JSON-LD

```
{
  "@context": "https://schema.org",
  "@type": "Course",
  "name": "Llenguatges de marques",
  "provider": {
    "@type": "EducationalOrganization",
    "name": "IES l'Estació"
  }
}
```

Problema

En JSON, «name» o «provider» no tenen un significat universal per si sols.

Context

JSON-LD associa les claus amb vocabularis i identificadors compartits.

Aplicació

Dades enllaçades i dades estructurades al web.

Activitat 5 · Quin format triaries?

Aplicació immediata del que acabem de veure

Decidiu el format principal per a: (1) una pàgina web accessible, (2) resposta d'una API, (3) pipeline de GitHub Actions, (4) configuració d'un projecte Python, (5) feed d'un podcast, (6) documentació d'un repositori. Justifiqueu cada decisió.

Temps

20 min

Format

Equips

Resultat

6 decisions justificades

Treball per a casa · Radiografia d'un repositori real

- Tria un repositori públic d'una aplicació web.
- Localitza almenys 4 fitxers de formats diferents: .html, .json, .yaml/.yml, .toml, .xml, .md...
- Per a cada fitxer: indica la funció, descriu l'estructura i copia un fragment curt.
- Explica per què creus que s'ha triat eixe format i no un altre.
- Tanca amb una comparació de 8–10 línies: «què aporta cada format al projecte?»

Lliurament

1–2 pàgines o README.md

Exit ticket · 5 minuts

1

Explica en una frase la diferència entre HTML i JSON.

2

Posa un cas real on XML continue sent una bona elecció.

3

Per què YAML no és «un XML modern»?

Mapa mental final

No hi ha un «millor format»: hi ha formats adequats per a problemes diferents.

HTML

Web
estructura +
semàntica

XML

Documents
estàndards +
validació

JSON

APIs
dades compactes

YAML / TOML

Configuració
DevOps + eines

Markdown

Documentació
text + Git

Llegir estructura → entendre el context → validar → transformar → intercanviar



Exemple opcional · Un SVG és XML que es converteix en una imatge

Obri el fitxer amb un editor de text i després amb el navegador.

EXEMPLE.SVG

```
<svg xmlns="http://www.w3.org/2000/svg"
  width="300" height="180" viewBox="0 0 300 180">
  <rect width="300" height="180" rx="18" fill="#eef5f8"/>
  <circle cx="90" cy="90" r="48" fill="#26a6b4"/>
  <rect x="160" y="50" width="95" height="80" rx="10" fill="#2b8c7e"/>
  <text x="150" y="165" text-anchor="middle">LMSGI · SVG</text>
</svg>
```



Prova guiada

Guarda com a exemple.svg → obri en VS Code → obri també en Chrome/Firefox. Canvia r, fill, width o text.

Exemple opcional · XML per a sindicació (Atom)

Un feed és un document XML que descriu contingut actualitzat periòdicament.

FEED.XML / ATOM.XML

```
<?xml version="1.0" encoding="utf-8"?>
<feed xmlns="http://www.w3.org/2005/Atom">
  <title>Notícies IES l'Estació</title>
  <link href="https://exemple.org/" />
  <updated>2026-09-12T12:00:00Z</updated>
  <id>https://exemple.org/feed</id>
  <entry>
    <title>Inici de curs</title>
    <link href="https://exemple.org/inici-curs" />
    <id>https://exemple.org/inici-curs</id>
    <updated>2026-09-12T10:00:00Z</updated>
    <summary>Informació per a l'alumnat.</summary>
  </entry>
</feed>
```

Què mirar

<feed> és l'arrel; <entry> representa una entrada; title, link, updated i id són dades estructurades.

Prova guiada

Guarda'l com feed.xml i obri'l en un navegador o editor. Identifica jerarquia i namespace.

Per aprofundir

Busca «Atom feed example RFC 4287» o usa el Feed Validator del W3C.

Exemple opcional · Markdown en un README.md de GitHub

El mateix fitxer és text pla en l'editor i document formatat quan GitHub el renderitza.

README.MD

Projecte LMSGI

Exemple per a **1r DAW**.

Posada en marxa

```
``bash
npm install
npm run dev
``
```

Tasques

- [x] Crear repositori
- [] Documentar l'API

Consulta [la documentació](docs/index.md).

Projecte LMSGI

Exemple per a 1r DAW.

POSADA EN MARXA

```
npm install
npm run dev
```

TASQUES

- ☒ Crear repositori
- ☐ Documentar l'API

Consulta la documentació.

Prova: crea un repositori → Add a README → edita README.md → compara la pestanya Code amb l'edició en brut.

Fitxers que poden obrir i inspeccionar sense haver estat a classe

La idea és que cada exemple tinga una ruta mínima: «on el trobe → amb què l'òbric → què he de mirar».

SVG

exemple.svg

Editor + navegador

Busca: <svg>, <rect>, <circle>, <text>.

Modifica atributs i comprova el resultat.

Atom / XML

feed.xml

Editor + navegador

Busca: <feed>, <entry>, namespace, dates i enllaços.

Markdown

README.md

GitHub + editor

Compara text font amb renderitzat: #, **, `` i llistes.

YAML / TOML

compose.yaml

pyproject.toml

VS Code

Observa jerarquia, indentació, claus, valors i arrays.

Fonts tècniques per mantindre el material actualitzat

HTML Living Standard

<https://html.spec.whatwg.org/>

XML 1.0 (W3C)

<https://www.w3.org/TR/xml/>

JSON · RFC 8259

<https://www.rfc-editor.org/rfc/rfc8259>

YAML 1.2.2

<https://yaml.org/spec/1.2.2/>

TOML 1.0.0

<https://toml.io/en/v1.0.0>

CommonMark

<https://spec.commonmark.org/>

JSON-LD 1.1 (W3C)

<https://www.w3.org/TR/json-ld/>